Generative AI – Assignment 2

Image Denoising with a Denoising Autoencoder and Face–Sketch Translation with CycleGAN

Question 1: Denoising Autoencoder (BSDS500)

Question 2: CycleGAN for Person Face Sketches with a Flask demo

Name: Muhammad Haroon Aziz

Roll No: i22-2697

Section: B

October 2026

Abstract

This report covers two tasks. In the first, a convolutional denoising autoencoder (DAE) was trained on 64×64 patches of the Berkeley Segmentation Dataset (BSDS500) to remove Gaussian and salt-and-pepper noise. The DAE improved PSNR by about 10 dB for Gaussian noise ($\sigma = 0.2$) and about 12 dB for salt-and-pepper noise (10%) over the noisy input, and the gain grew with the noise level. Larger bottlenecks gave better scores, with diminishing returns. In the second task, a CycleGAN was trained on unpaired photos and sketches from the Person Face Sketches dataset at 128×128 resolution for 30 epochs. Photo-to-sketch translation gave clean line drawings. Sketch-to-photo translation produced face-like images with blur, similar warm colours and some distorted features. A Flask web application with upload, camera capture and automatic sketch/photo detection demonstrates the trained generators.

Contents

1 Question 1: Image Denoising with a Denoising Autoencoder **2**

1.1 Dataset and preparation . . . 2
1.2 Noise injection . . . 2
1.3 Model architecture . . . 3
1.4 Training . . . 3
1.5 Evaluation metrics . . . 3
1.6 Main results . . . 4
1.7 Experiment A: effect of noise level . . . 5
1.8 Robustness to unseen noise levels . . . 6
1.9 Experiment B: effect of bottleneck size . . . 7
1.10 Limitations . . . 8

2 Question 2: CycleGAN for Person Face Sketches **8**

2.1 Dataset . . . 8
2.2 Method . . . 9
2.3 Architecture . . . 9
2.4 Training setup . . . 9
2.5 Training behaviour . . . 10
2.6 Results on held-out test images . . . 11
2.7 Hyperparameters and decisions . . . 12
2.8 User interface (Flask) . . . 12
2.9 Limitations . . . 12

3 Conclusion **12**

4 Reproducibility **13**

1 Question 1: Image Denoising with a Denoising Autoencoder

1.1 Dataset and preparation

BSDS500 was downloaded from the Berkeley project page and its official split was used: 200 training, 100 validation and 200 test images. The split is by image, so no test patch comes from a training image. Images were normalised to $[0, 1]$ by dividing by 255. Because 500 images is small, random 64×64 patches were cropped: 30 per training image (6,000 patches, re-sampled every epoch), and 10 per validation/test image at fixed positions (1,000 and 2,000 patches) so that evaluation is reproducible.

Table 1: Dataset splits and patch counts.

Split	Images	Patches	Crop positions
Train	200	6,000	Random each epoch
Validation	100	1,000	Fixed (seed 1)
Test	200	2,000	Fixed (seed 2)

1.2 Noise injection

- **Gaussian:** $x_{\text{noisy}} = \text{clip}(x + \mathcal{N}(0, \sigma^2), 0, 1)$ with $\sigma \in \{0.05, 0.1, 0.2, 0.3\}$.
- **Salt-and-pepper:** a fraction $p \in \{5, 10, 20, 30\}\%$ of pixels is corrupted, half set to 0 and half to 1, at the same location in all three channels.

Training noise is generated freshly for every batch (a form of augmentation); validation and test noise use fixed seeds. A sanity check gave an MSE of 0.032 for $\sigma = 0.2$ (slightly below $\sigma^2 = 0.04$ because of clipping) and 9.8% changed pixels for 10% salt-and-pepper.

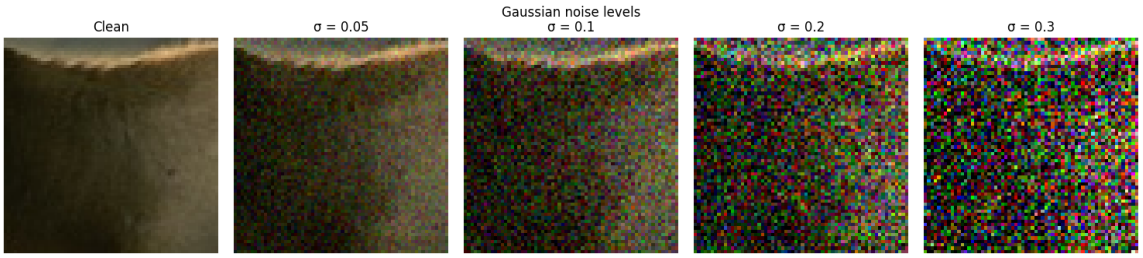


Figure 1: Clean patch with Gaussian noise at $\sigma = 0.05, 0.1, 0.2, 0.3$.

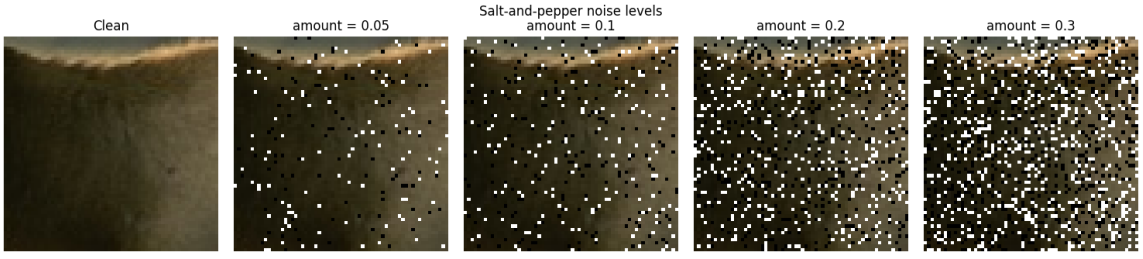


Figure 2: Clean patch with salt-and-pepper noise at 5, 10, 20 and 30%.

1.3 Model architecture

The encoder has three stride-2 convolutions with ReLU, reducing $3 \times 64 \times 64$ to a $C \times 8 \times 8$ bottleneck; the decoder mirrors it with three stride-2 transposed convolutions. ReLU is used in all hidden layers, and a Sigmoid on the output keeps reconstructions in $[0, 1]$.

Table 2: DAE architecture (C = bottleneck channels, default 64).

Stage	Layer	Output	Activation
Input	–	$3 \times 64 \times 64$	–
Encoder 1	Conv 3×3, s2, 32	$32 \times 32 \times 32$	ReLU
Encoder 2	Conv 3×3, s2, 64	$64 \times 16 \times 16$	ReLU
Bottleneck	Conv 3×3, s2, C	$C \times 8 \times 8$	ReLU
Decoder 1	ConvT 4×4, s2, 64	$64 \times 16 \times 16$	ReLU
Decoder 2	ConvT 4×4, s2, 32	$32 \times 32 \times 32$	ReLU
Output	ConvT 4×4, s2, 3	$3 \times 64 \times 64$	Sigmoid

With $C = 64$ the bottleneck holds $64 \cdot 8 \cdot 8 = 4{,}096$ values against $3 \cdot 64 \cdot 64 = 12{,}288$ input values (3:1 compression) and the model has 156,259 parameters. Other sizes tested: $C = 8, 16, 128$ (512, 1,024, 8,192 values).

1.4 Training

The model reconstructs the clean patch from its noisy version using MSE loss, Adam (learning rate 10^{-3}), batch size 64 and 30 epochs, keeping the weights with the lowest validation loss. A separate model was trained for each noise type, level and bottleneck size (about 27 s each on a Tesla T4). Training and validation losses fell together and flattened with no visible gap, so there was no sign of overfitting.

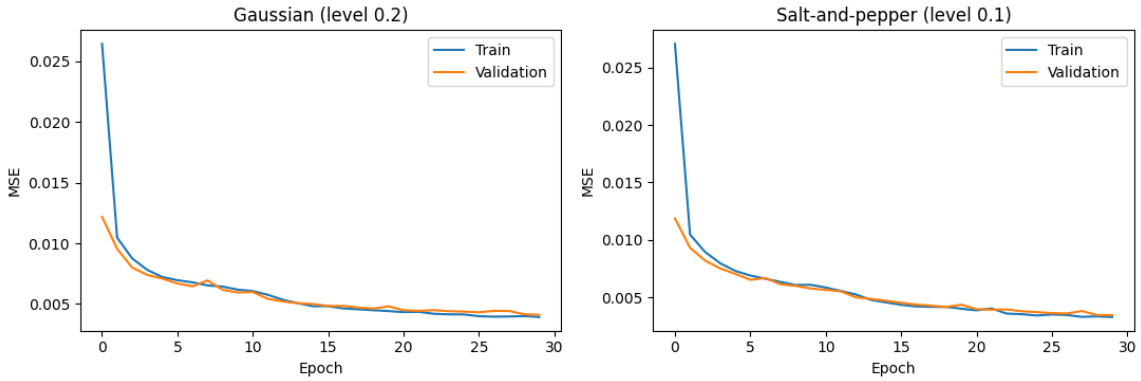


Figure 3: Training and validation MSE for the Gaussian ($\sigma = 0.2$) and salt-and-pepper (10%) models.

1.5 Evaluation metrics

Models were evaluated on the 2,000 test patches with MSE, PSNR ($10 \log_{10}(1/\text{MSE})$ per image, averaged) and SSIM (PyTorch implementation, 11×11 Gaussian window, $\sigma = 1.5$). On 20 images it matched scikit-image exactly (0.1861 for both).

1.6 Main results

Table 3: Test-set results of the default models (bottleneck 64).

Noise	Input	MSE	PSNR (dB)	SSIM
Gaussian $\sigma = 0.2$	Noisy	0.0329	14.86	0.2019
Gaussian $\sigma = 0.2$	DAE output	0.0042	25.16	0.6578
Salt-and-pepper 10%	Noisy	0.0318	15.02	0.2628
Salt-and-pepper 10%	DAE output	0.0035	26.81	0.7385

The DAE raised PSNR by 10.3 dB (Gaussian) and 11.8 dB (salt-and-pepper) and roughly tripled SSIM. Denoised patches are much cleaner than the inputs. Smooth regions are recovered well (about 30–35 dB), but a patch with fine texture (a town scene) reached only about 18 dB, because the compressed bottleneck removes noise partly by blurring detail.

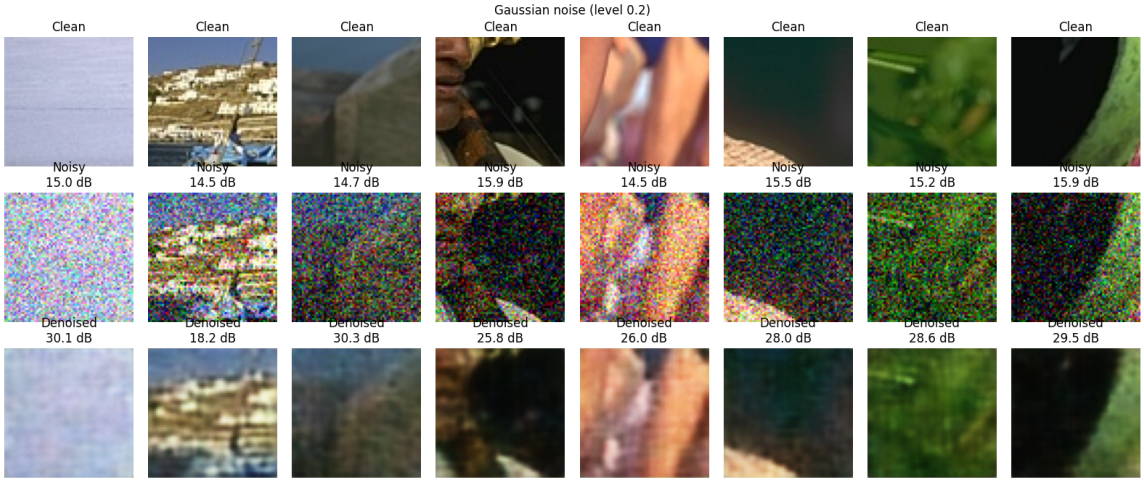


Figure 4: Clean, noisy and denoised patches, Gaussian noise ($\sigma = 0.2$), with per-patch PSNR.

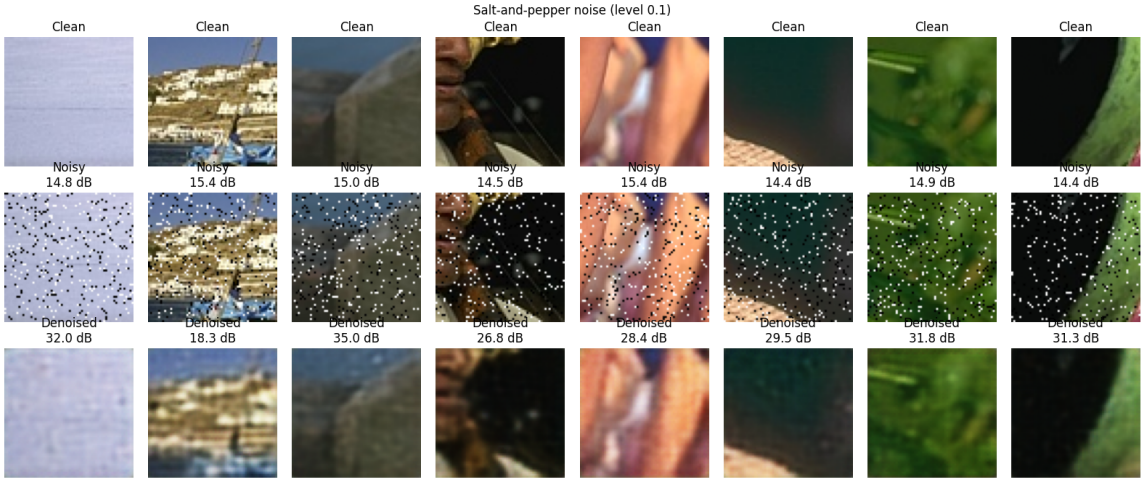


Figure 5: Clean, noisy and denoised patches, salt-and-pepper noise (10%).

1.7 Experiment A: effect of noise level

A separate model (bottleneck 64) was trained and tested at each level.

Table 4: Test PSNR (dB) and SSIM per noise type and level.

Noise	Level	Noisy PSNR	DAE PSNR	Gain	Noisy SSIM	DAE SSIM
Gaussian	0.05	26.21	27.75	1.54	0.6364	0.7813
Gaussian	0.10	20.37	26.83	6.46	0.3960	0.7357
Gaussian	0.20	14.86	25.16	10.30	0.2019	0.6578
Gaussian	0.30	11.99	24.31	12.32	0.1251	0.6193
Salt-and-pepper	0.05	18.03	27.56	9.53	0.4336	0.7701
Salt-and-pepper	0.10	15.02	26.81	11.79	0.2628	0.7385
Salt-and-pepper	0.20	12.00	26.00	14.00	0.1425	0.6937
Salt-and-pepper	0.30	10.24	25.25	15.01	0.0931	0.6521

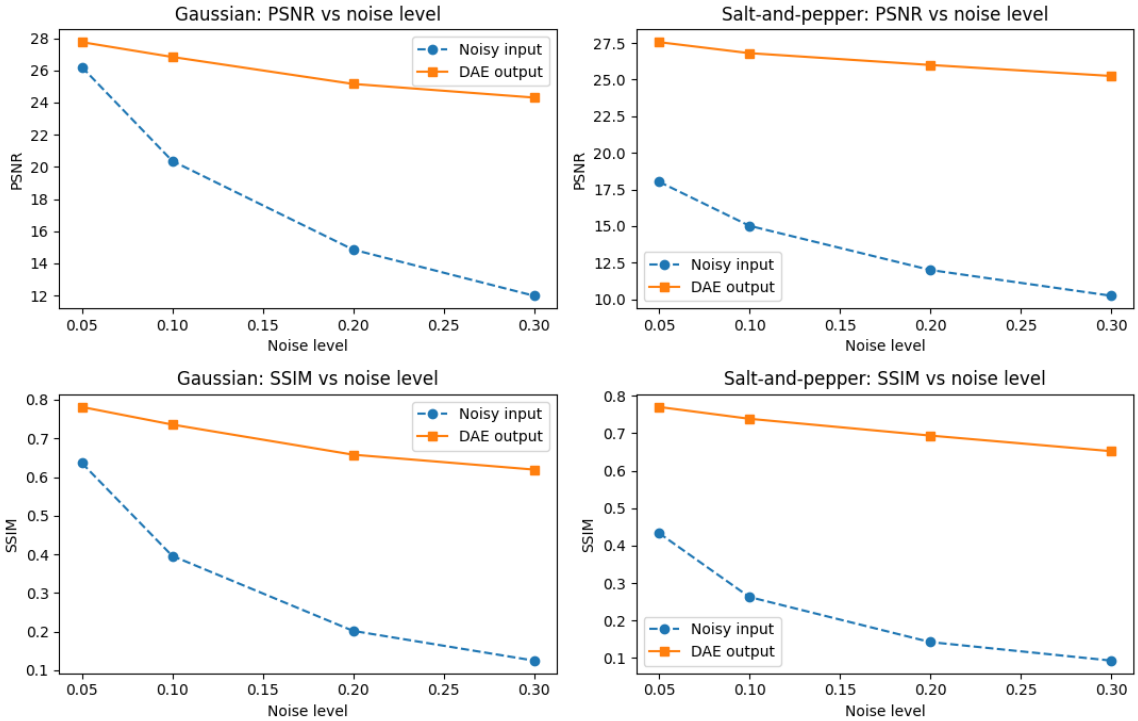


Figure 6: PSNR and SSIM against noise level for the noisy input and the DAE output.

- Performance falls as noise rises: Gaussian DAE PSNR drops from 27.75 to 24.31 dB ($\sigma = 0.05 \rightarrow 0.3$) while the noisy input drops from 26.21 to 11.99 dB; salt-and-pepper drops from 27.56 to 25.25 dB.
- The gain over the noisy input grows with noise: 1.5 to 12.3 dB (Gaussian) and 9.5 to 15.0 dB (salt-and-pepper).
- At very low Gaussian noise the gain is small, since the input is already good and bottleneck blur limits PSNR to about 27.8 dB.
- Salt-and-pepper noise was not harder than Gaussian noise for this model; impulse noise leaves most pixels exact, so corrupted pixels are easy to detect.

1.8 Robustness to unseen noise levels

The default models (trained at $\sigma = 0.2$ and 10%) were tested at all levels.

Table 5: Models trained at one level, tested at all levels.

Noise	Trained at	Test level	MSE	PSNR (dB)	SSIM
Gaussian	0.2	0.05	0.0039	25.76	0.6988
Gaussian	0.2	0.10	0.0039	25.67	0.6954
Gaussian	0.2	0.20	0.0042	25.16	0.6578
Gaussian	0.2	0.30	0.0060	22.93	0.5529
Salt-and-pepper	0.1	0.05	0.0034	27.13	0.7570
Salt-and-pepper	0.1	0.10	0.0035	26.81	0.7385
Salt-and-pepper	0.1	0.20	0.0042	25.34	0.6740
Salt-and-pepper	0.1	0.30	0.0057	23.35	0.5764

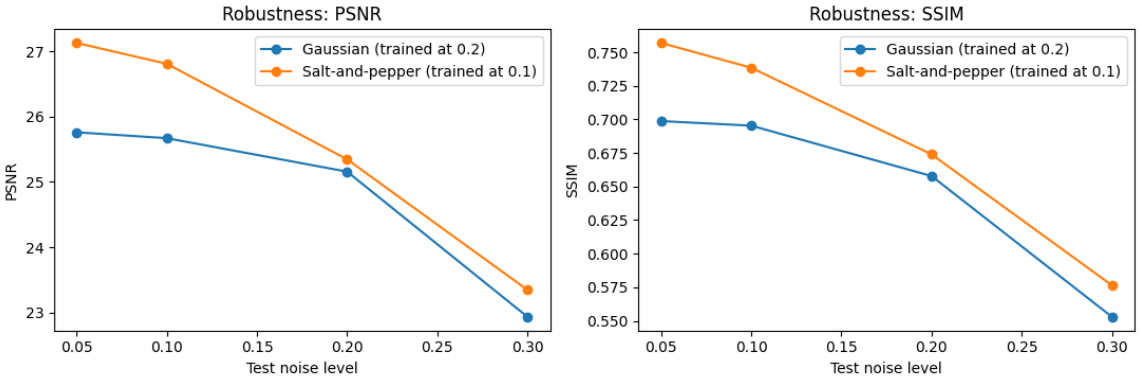


Figure 7: Robustness of the default models to different test noise levels.

Both models degrade little on lighter noise but drop clearly on heavier noise than they were trained on. At the heaviest level the matched model scored 24.31 dB (Gaussian) and 25.25 dB (salt-and-pepper) versus 22.93 and 23.35 dB for the default models, so matching the training level is worth about 1.4 and 1.9 dB. The $\sigma = 0.2$ model scores higher on $\sigma = 0.05, 0.1$ only because lighter noise is easier.

1.9 Experiment B: effect of bottleneck size

Table 6: Effect of bottleneck size on test performance.

Noise	Channels	Latent	Compr.	Params	PSNR (dB)	SSIM
Gaussian 0.2	8	512	24:1	66,603	24.28	0.6088
Gaussian 0.2	16	1,024	12:1	79,411	24.88	0.6356
Gaussian 0.2	64	4,096	3:1	156,259	25.16	0.6578
Gaussian 0.2	128	8,192	1.5:1	258,723	25.64	0.6863
S&P 10%	8	512	24:1	66,603	25.33	0.6457
S&P 10%	16	1,024	12:1	79,411	26.02	0.6931
S&P 10%	64	4,096	3:1	156,259	26.81	0.7385
S&P 10%	128	8,192	1.5:1	258,723	26.96	0.7465

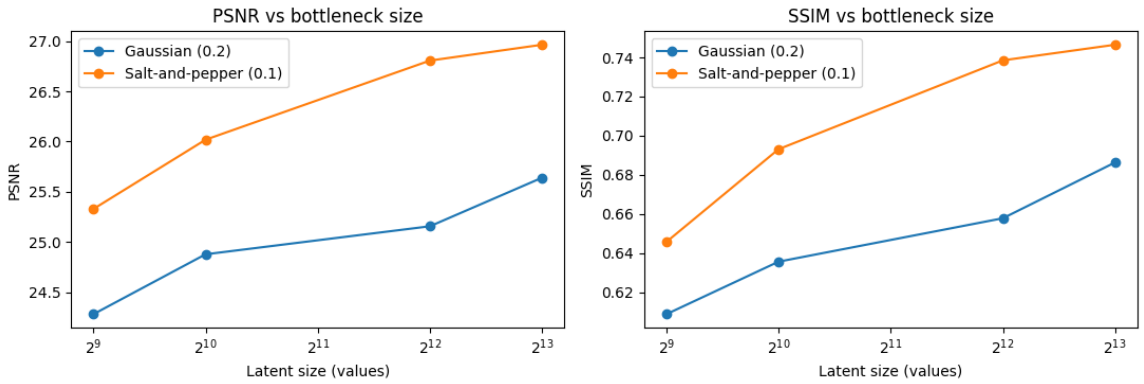


Figure 8: PSNR and SSIM against latent size.

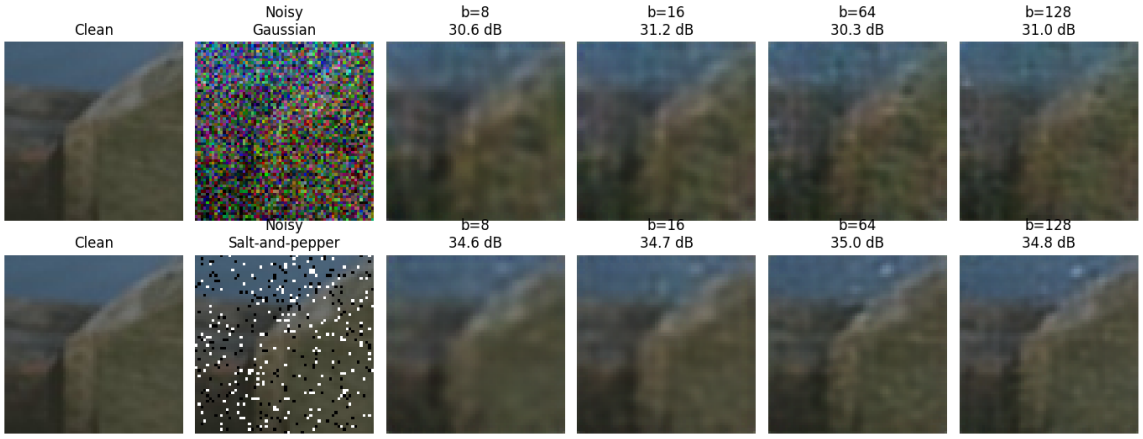


Figure 9: The same test patch denoised with each bottleneck size.

PSNR and SSIM rise steadily with bottleneck size, with diminishing returns: for salt-and-pepper noise, doubling the latent size from 4,096 to 8,192 adds only 0.16 dB, while 512 to 1,024 adds 0.69 dB (Gaussian: 0.48 dB at the top). The worry that a very large bottleneck might learn an identity mapping was not observed: even the largest model (1.5:1) clearly beat the noisy input.

Larger bottlenecks were not tested. The single patch in Figure 9 differs by under 1 dB between sizes and is not strictly ordered, so conclusions rest on the test-set tables.

1.10 Limitations

Each setting was trained once with a single seed, so differences of a few hundredths of a dB are not meaningful. Only 64×64 patches and 30 epochs were used (losses were still falling slowly). The DAE blurs fine texture; a deeper or skip-connected network such as a U-Net would likely keep more detail.

2 Question 2: CycleGAN for Person Face Sketches

2.1 Dataset

The Person Face Sketches dataset (Kaggle, almightyj) has train, validation and test folders, each with `photos` and `sketches`. The 512×512 images come in pairs with matching names, but CycleGAN does not use the pairing: photos (domain A) and sketches (domain B) were sampled independently at random.

Table 7: Dataset folders and the subset used.

Folder	Images per domain	Used here
Train	20,655	2,000 random per domain
Validation	1,000	Not used
Test	679	200 random per domain

Images were resized to 128×128, normalised to $[-1, 1]$ and randomly flipped. The subset and size were chosen for GPU-time reasons: a first test with 4,000 images per domain took about 486 s per epoch, while 2,000 images took about 229 s.

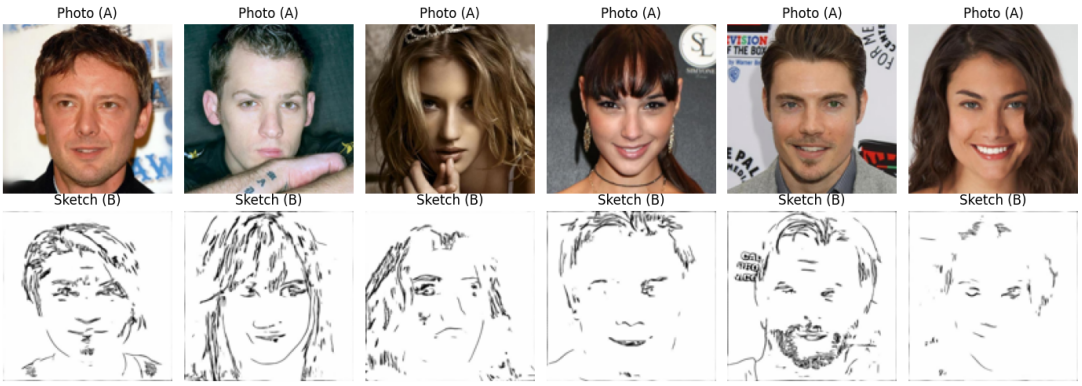


Figure 10: Preview of the training data: photos (top) and sketches (bottom). The two rows show different people because the data is unpaired.

2.2 Method

CycleGAN learns $G : A \to B$ (photo to sketch) and $F : B \to A$ (sketch to photo) with discriminators D_A and D_B. As in the original paper:

$$\begin{aligned}
\mathcal{L}_{\text{GAN}}(G, D_B) &= \mathbb{E}_b[(D_B(b) - 1)^2] + \mathbb{E}_a[D_B(G(a))^2] \quad \text{(least-squares GAN)}, \\
\mathcal{L}_{\text{cyc}} &= \mathbb{E}_a\|F(G(a)) - a\|_1 + \mathbb{E}_b\|G(F(b)) - b\|_1, \\
\mathcal{L}_{\text{id}} &= \mathbb{E}_b\|G(b) - b\|_1 + \mathbb{E}_a\|F(a) - a\|_1, \\
\mathcal{L} &= \mathcal{L}_{\text{GAN}}(G, D_B) + \mathcal{L}_{\text{GAN}}(F, D_A) + 10\,\mathcal{L}_{\text{cyc}} + 5\,\mathcal{L}_{\text{id}}.
\end{aligned}$$

2.3 Architecture

- **Generator (ResNet):** 7×7 conv (64) $\to$ two stride-2 convs (128, 256) $\to$ 6 residual blocks $\to$ two stride-2 transposed convs (128, 64) $\to$ 7×7 conv (3) $\to$ Tanh, with instance normalisation and reflection padding. 7,837,699 parameters each.
- **Discriminator (70×70 PatchGAN):** four 4×4 convs (64, 128, 256, 512) with LeakyReLU(0.2) and instance norm, then a one-channel map (14×14 for a 128×128 input). 2,764,737 parameters each.

2.4 Training setup

Table 8: Hyperparameters and training setup.

Setting	Value
Image size	128×128
Training images	2,000 per domain
Batch size	4 (500 steps/epoch)
Optimizer	Adam, $\beta_1 = 0.5$, $\beta_2 = 0.999$
Learning rate	2×10^{-4} for 15 epochs, then linear decay to 0
Epochs	30
$\lambda_{\text{cyc}}/\lambda_{\text{id}}$	10 / 5
Residual blocks	6
Image buffer	50 past generated images
Hardware	Kaggle Tesla T4, about 229 s/epoch (about 1.9 h)

A checkpoint with both generators, both discriminators, optimisers, schedulers, loss history and epoch number was saved after every epoch (to a temporary file, then renamed, so a crash cannot corrupt it). On start-up the notebook loads the latest checkpoint and continues from the last finished epoch, as the assignment requires.

2.5 Training behaviour

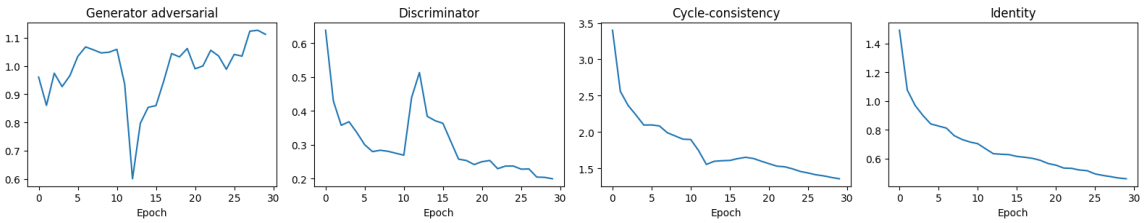


Figure 11: Generator adversarial, discriminator, cycle-consistency and identity losses per epoch.

- Cycle-consistency loss fell from 3.40 to 1.36 and identity loss from 1.49 to 0.46, steadily and without NaN values.
- Around epochs 12–13 the discriminator briefly weakened (loss rose from about 0.27 to 0.51) while the generator adversarial loss dropped to 0.60; both recovered within a few epochs, which is common in GAN training.
- In the last epochs the generator adversarial loss was about 1.1 and the discriminator loss about 0.20, flattening as the learning rate decayed. Quality was judged from the images, not from these GAN losses.

Figures 12–15 show the sample grids at epochs 1, 10, 20 and 30 (rows: real photo, fake sketch, reconstructed photo, real sketch, fake photo, reconstructed sketch).

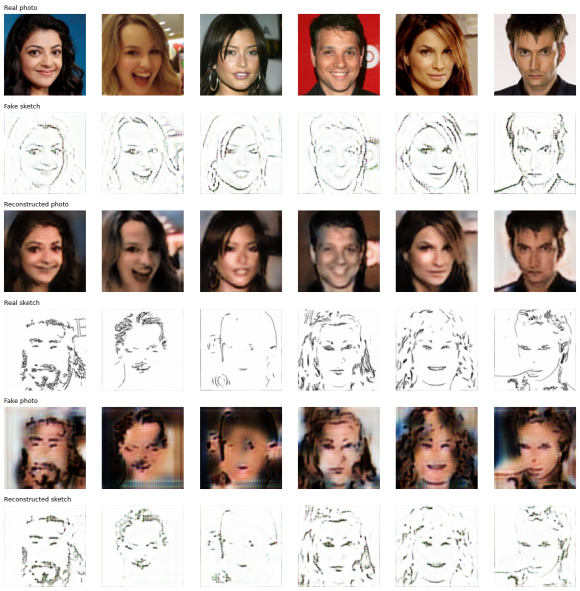


Figure 12: Epoch 1.

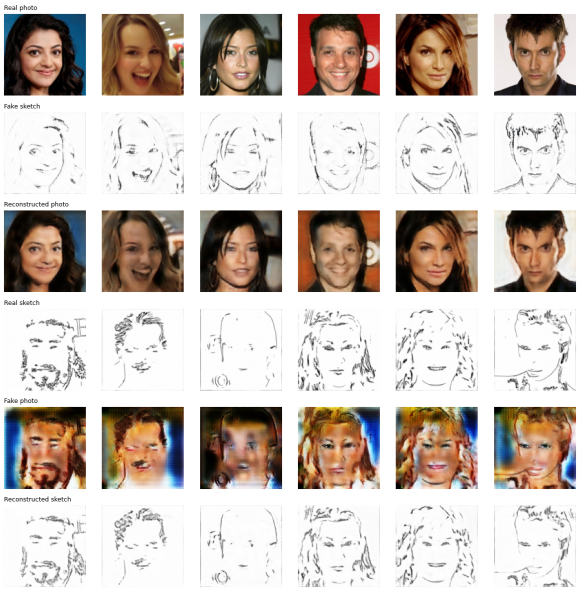


Figure 13: Epoch 10.

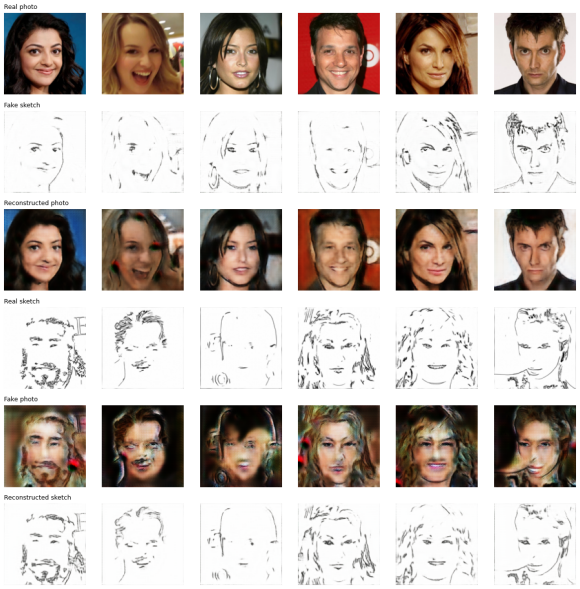


Figure 14: Epoch 20.

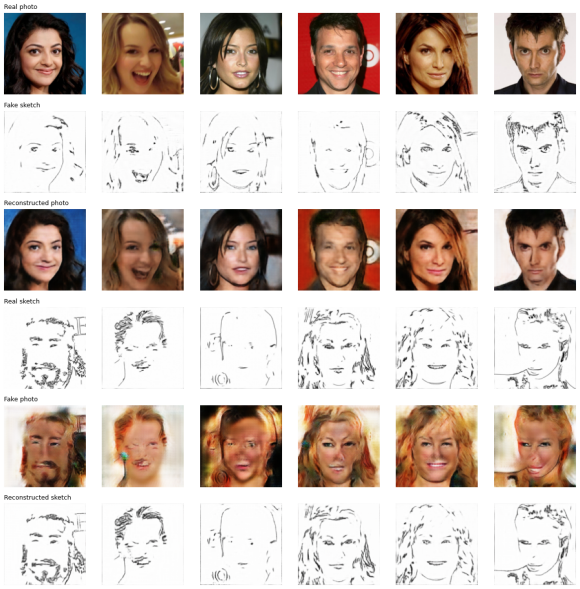


Figure 15: Epoch 30.

2.6 Results on held-out test images

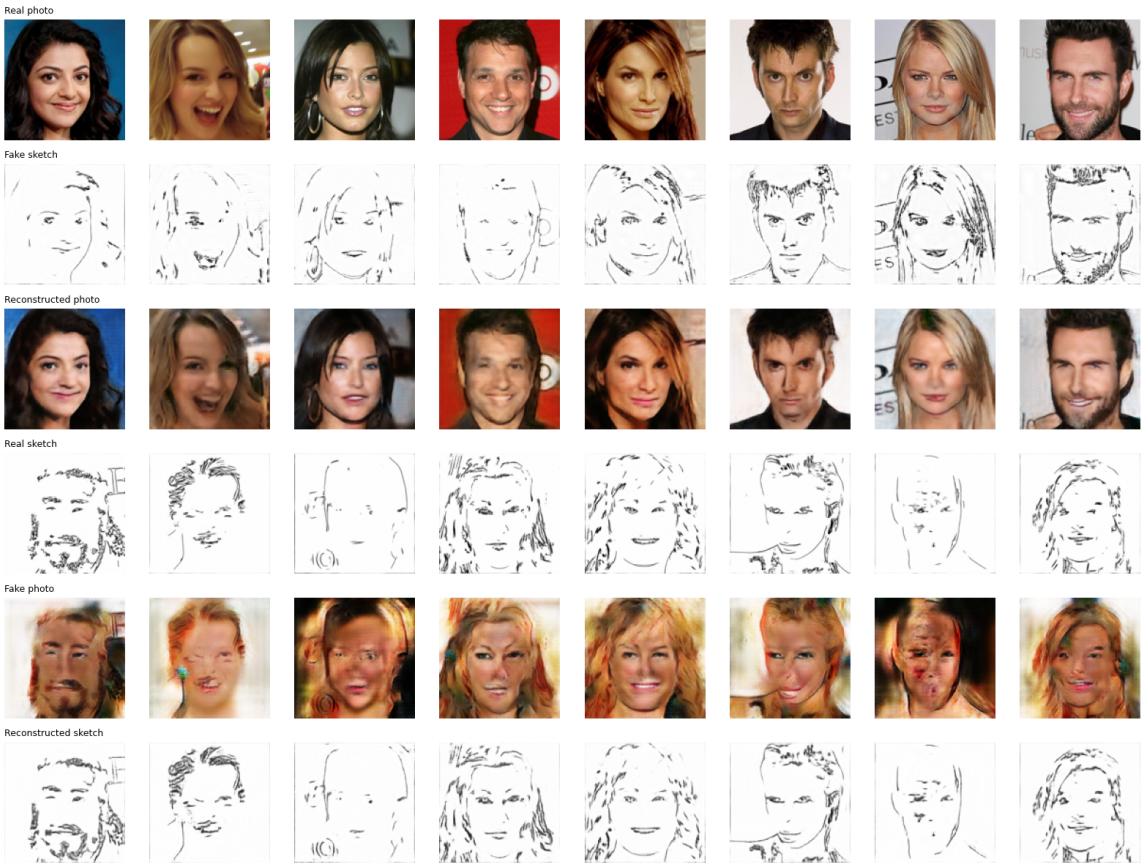


Figure 16: Test results. Rows: real photo, generated sketch, reconstructed photo, real sketch, generated photo, reconstructed sketch.

- **Photo to sketch works well.** Generated sketches are clean black line drawings on white backgrounds, similar to the dataset style, and follow the hair, eyes, nose and mouth of the

input.

- The cycle reconstructions (rows 3 and 6) are close to the originals, so content survives the round trip.
- **Sketch to photo is weaker.** Outputs are face-like but blurry, with similar warm skin tones across different inputs and some distorted eyes or mouths. A sketch carries only outlines, so the generator must invent colour and texture, and 30 epochs at 128×128 on 2,000 images was not enough to do this convincingly.

2.7 Hyperparameters and decisions

Standard CycleGAN values were kept (learning rate 2×10^{-4}, $\beta_1 = 0.5$, $\lambda_{\text{cyc}} = 10$, $\lambda_{\text{id}} = 5$, buffer 50). The choices made were practical: 128×128 resolution with 6 residual blocks (the paper's setting for 128×128), 2,000 training images per domain instead of 20,655, and 30 epochs instead of the 60 originally planned, after measuring the time per epoch. A 2-epoch test run came first to check the code and measure speed. No systematic comparison of learning rates or batch sizes was run, so no claims are made about their effect.

2.8 User interface (Flask)

A Flask web application loads both generator weight files once at start-up and runs inference on the CPU. The page allows the user to upload an image or capture one from the camera (`getUserMedia`); the type is detected automatically and converted in the right direction, or chosen manually with a dropdown (auto, force face→sketch, force sketch→face); input and output are shown side by side with a status line such as "Detected: Real face → Face → Sketch".

Detection uses colour saturation: mean$|R - G|$ plus mean$|G - B|$. On 300 images of each type sketches scored exactly 0.0 and photos averaged 45.1 (minimum 1.5). An image below the threshold of 5.0 is treated as a sketch and sent to the sketch-to-face generator; otherwise it goes to the face-to-sketch generator. To run: install `requirements.txt`, run `python app.py` inside `flask_app` and open `http://127.0.0.1:5000`.

2.9 Limitations

- Sketch-to-photo quality is limited (blur, similar colours, distorted features) because of short training, small images and a data subset.
- The saturation test can fail for black-and-white photos (detected as sketches) and for tinted or photographed hand drawings (detected as photos); the manual override handles these.
- The model expects close-up, front-facing face crops like the dataset; other images give poor results.
- Output is 128×128 enlarged for display, so it looks soft. Only one training run was done.
- Possible improvements: more epochs (training can resume from the saved checkpoint), 256×256 images with 9 residual blocks, and the full dataset.

3 Conclusion

The denoising autoencoder removes both Gaussian and salt-and-pepper noise well, with larger gains for heavier noise and better scores for larger bottlenecks, at the cost of blurred fine detail. The CycleGAN learned a good face-to-sketch mapping and a weaker sketch-to-face mapping,

both available through the Flask application. All numbers in this report come from the submitted notebooks.

4 Reproducibility

- Both notebooks were run on Kaggle with a Tesla T4 GPU (PyTorch 2.11, CUDA 12.8). Q1 downloads BSDS500 from the Berkeley page; Q2 uses the Kaggle dataset `almightyj/person-face-sketch`
- DataLoaders use `num_workers=0` because multi-worker loading failed in this environment.
- Q2 checkpoints resume automatically from `last.pth`. The Flask app needs flask, torch, torchvision, pillow and numpy.